*Zero Edoardo, SMID*

**Laboratorio 5 : Neural Networks**

In questo laboratorio sono state studiate le reti neurali sia dal punto di vista teorico sia da quello pratico, sono state utilizzate per implementare e risolvere un problema di riconoscimento di immagini di cifre .
Il training set utilizzato per addestrare le reti neurali contiene 60000 immagini ognuna composta da 28x28 pixel rappresenta un numero, ciascun pixel ha un valore nella scala *greyscale* ad indicare il colore.
La struttura della rete neurale progettata per questo compito prevede tre layer di neuroni: il primo ha 784 neuroni di input, uno per ogni pixel, il secondo ha un numero di neuroni da stabilire e lo si indica con $n_h$ , ed infine l'ultimo strato sarà quello di output che conterrà 10 neuroni , uno per ogni cifra, quello con l'output più alto indicherà la cifra predetta dalla rete neurale, quindi , l'output sarà un vettore di dimensione 10 che avrà tutti zeri ad eccezione di un 1 nel posto della cifra predetta dalla rete.
Lo scopo è quello di trovare un insieme di *bias* e *pesi* che meglio approssimano la predizione, minimizzando la funzione costo della rete, per far ciò si utilizza la tecnica SGD definita dalla funzione *train_network_SGD* che è il cuore del processo di addestramento della rete neurale.
La funzione prende in input:

- *Sizes :* il numero di neuroni di ciascun livello della rete;
- *Training data :* le tuple (X,Y) con X l'input e Y l'output desiderato;
- *Test Data :* serve per calcolare la precisione con la funzione che calcola l'errore;
- *Epochs :* il numero di volte che il dataset verrà processato;
- *Mini-batch size:* il numero di esempi di training da includere in ciascun mini batch
- *Learning_rate :* il tasso di apprendimento nel processo SGD

Inizialmente i pesi e il bias vengono inizializzati casualmente. Durante ogni epoca i dati di addestramento vengono rimescolati e suddivisi in mini-batch. Per ciascuna mini-batch la funzione *update_mini_batch* calcola i gradienti tramite *backpropagation* e aggiorna i pesi e bias usando il tasso di apprendimento. Alla fine di ogni epoca , la funzione valuta le prestazioni della rete sul dataset di test, calcolando il numero di campioni classificati correttamente. Al termine dell'addestramento , restituisce i pesi e bias finali, insieme ai campioni di test classificati erroneamente.
Dopo aver definito le funzioni necessarie per l'addestramento della rete e quella per riformattare i set di dati , si è iniziato l'addestramento della rete.
Quindi assumendo 10 neuroni nello strato nascosto si è allenato il modello, producendo come output il livello di precisione per ogni epoca:

```
Epoch 1/5. Accuracy: 0.8714
Epoch 2/5. Accuracy: 0.8871
Epoch 3/5. Accuracy: 0.8897
Epoch 4/5. Accuracy: 0.9013
Epoch 5/5. Accuracy: 0.8986
```

Il modello migliora la precisione ogni epoca , in particolar modo nelle prime tre , poi tende a stabilizzarsi e non crescere più monotonamente , oscillando tra valori circa al 90% .

In seguito è stato studiato il comportamento della rete al variare del :

1. numero di neuroni nello strato nascosto;
2. numero di strati nascosti;
3. tasso di apprendimento;
4. grandezza del mini_batch;
5. numero di epoche;

Nel primo caso l'idea è stata quella di definire una lista di $n_h$ =[5,10,20,30,50] e attraverso un ciclo *for* sono state calcolate le precisioni su ogni valore per ogni epoca , osservando i risultati , i valori peggiori si sono visti per $n_h$ =5,50 , per gli altri valori la precisione è sempre stata alta in ogni epoca a maggior ragione nei casi di $n_h$ =20,30 dove il valore superava il 90% ogni epoca, tuttavia una numerosità maggiore nello strato

nascosto comporta un tempo maggiore di elaborazione . Quindi dovendo scegliere un $n_h$ minimo per avere valori accettabili di precisione è stato deciso $n_h$ =10.
Successivamente è stato modificato il numero di strati nascosti , in 2 e 3 , all'interno della rete neurale per verificare se il numero di neuroni per layer trovato in precedenza sarebbe stato modificato. Il comportamento è simile al caso precedente perciò dal punto di vista di costo computazionale e tempo è meglio considerare un solo strato, il compito di classificare le cifre è abbastanza semplice quindi è sufficiente considerare un solo strato nascosto con 10 neuroni.
A seguito dello studio del comportamento della rete al variare del tasso di apprendimento nella SGD è stato osservato che , per valori eccessivamente troppo alti o troppo bassi, la percentuale di predizioni corrette rimane molto bassa , pur registrando qualche miglioramento di epoca in epoca. Ciò accade perché , se il tasso è troppo alto, gli aggiornamenti sono troppo grandi e il modello salta il minimo della funzione di costo, rendendo la convergenza instabile. Mentre se il tasso è troppo piccolo , gli aggiornamenti sono così ridotti da rallentare il processo di apprendimento impedendo al modello di migliorare significativamente.
Per quanto riguarda la grandezza delle *mini_batch* si sono scelte le grandezze : [1,5,10, 100, 300, 500] per valori piccoli fino a 100 è risultato che la rete ha lavorato bene , mantenendo la precisione sopra l'80% . Per il valore di 300 la precisione di predizione alla quinta epoca scende al 74% e per 500 sotto al 60%. In generale con l'aumentare della grandezza dei *mini_batch* ci si aspetta che la precisione diminuisca perché gli aggiornamenti sono meno frequenti , rallentando la capacità della rete di convergere a una soluzione ottimale, se però si aumentassero il numero di epoche , ad esempio fino a 50 per *mini_batch_size = 500* , la rete passando più volte sul dataset effettua molte iterazioni ma apprende , quindi si ottiene una buona predizione ma è molto lenta.
Infine fissando i valori di *mini_batch_size , learning_rate,* $n_h$ e *un solo strato nascosto*, si è fatto variare il numero di epoche , ovviamente all'aumentare delle epoche la precisione della rete migliora in quanto il modello viene addestrato ripassando più volte il dataset.